

Production-grade LLM inference



Daria Balashova
ML Engineer



Gleb Berjoskin
ML Solutions Architect

Section 0

Intro

That was a bait

- What does “production” mean?
- What are our business metrics?
- What are our target SLOs?
- Any other success criteria?
- What do we know about traffic patterns?
- ...



Agenda

- Layers of a production inference system
- Each layer: role and design principles
- Making the layers work together
- Designing for robustness and maintainability

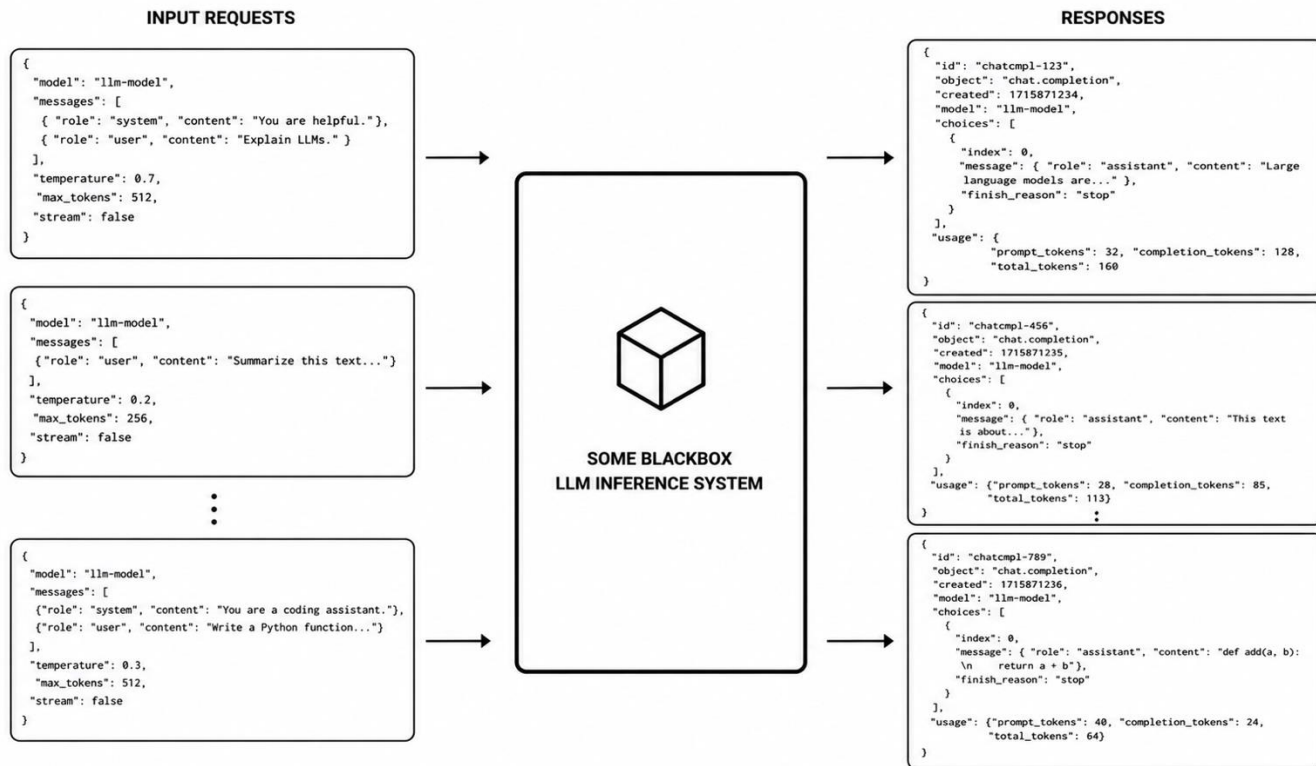
Few words on business metrics

- We want to get max revenue per operating GPU
- This usually means utilizing GPU fleet in the most efficient way
- Efficient here means max possible throughput at min possible latency
- Points above strictly define the shape of prod-grade LLM inference systems

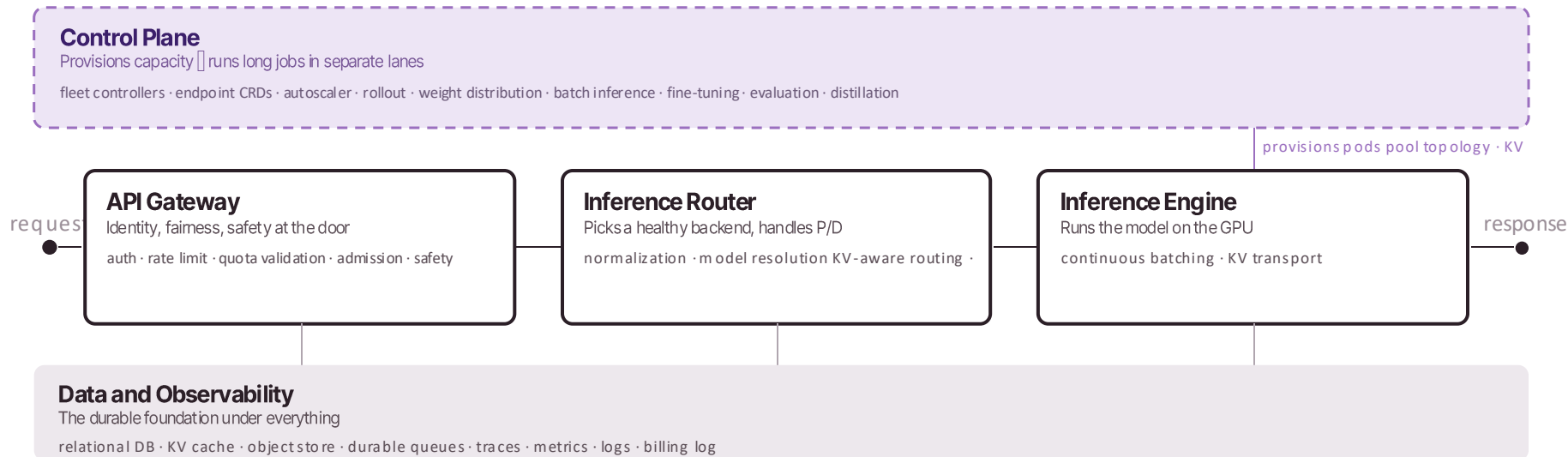
Section 1

Layers of prod-grade inference system

Setting the stage

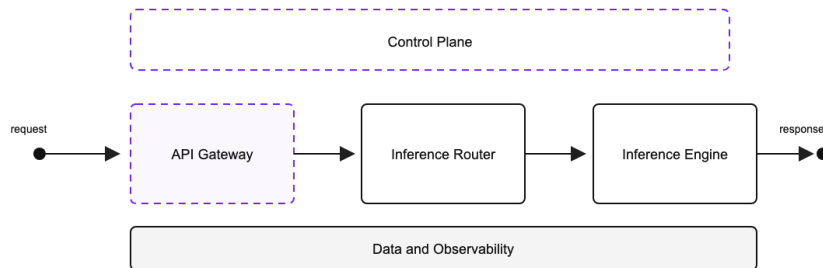


What's inside that black box



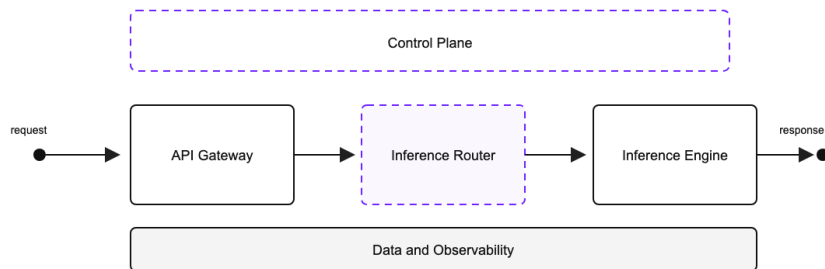
Api gateway

- Every request meets this layer first
- This layer ensures we know who sent the request and thus can determine wheether to process it
- This is especially important since LLMs are expensive to run



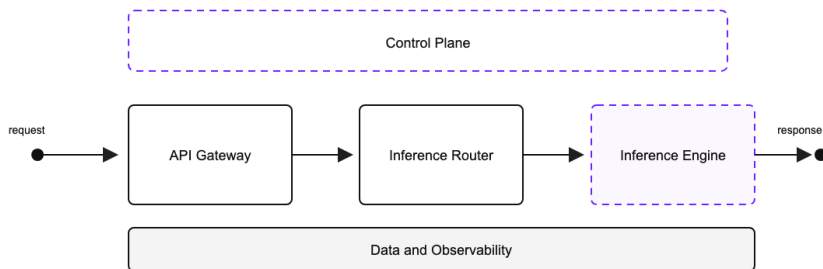
Inference router

- Model deployment may use replicas in multiple regions, on different hardware, any of which can be unhealthy at any moment.
- Router tracks all of these and does a number of fancy optimizations while directing user request to a specific inference engine deployment (e.g. KV-aware routing).



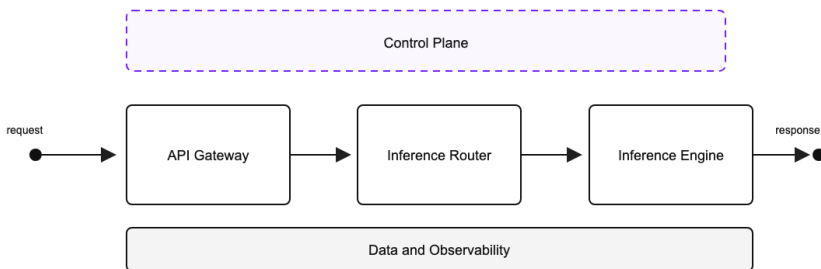
Inference engine

- The layer that actually runs LLMs on GPUs
- This layer is responsible for most of fancy inference optimizations ppl are talking about: continuous batching, speculative decoding, compression-related speedups, custom kernels, etc.
- Some of optimizations require cross-pod synchronization (e.g. P/D) and inference engine takes care of preparations for that.



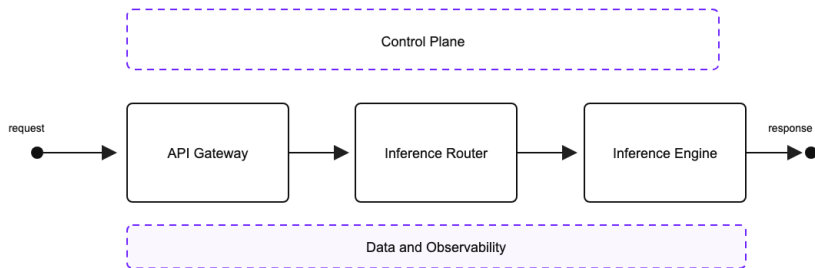
Control plane

- The layer manages environment that processes incoming requests.
- Managing hardware, communications, autoscaling, rollouts and many more responsibilities belong here.
- This layer also takes care of long-running jobs like high-throughput-oriented batch processing workloads.



Data and observability layer

- This layer keeps track of everything that happens making the whole system transparent, recoverable, and debuggable.
- Typically operates with several stores used for different types of information.



Questions?

Section 2

Zooming in

API gateway: a shallow dive

Responsibility

1. Contract and validation

2. Identity and authorization

3. Rate and concurrency limits

4. Quota admission

5. Safety and content filters

Technologies / state

OpenAPI / JSON Schema
payload size + model validation

JWT + JWKS
IAM / ACL service + local cache

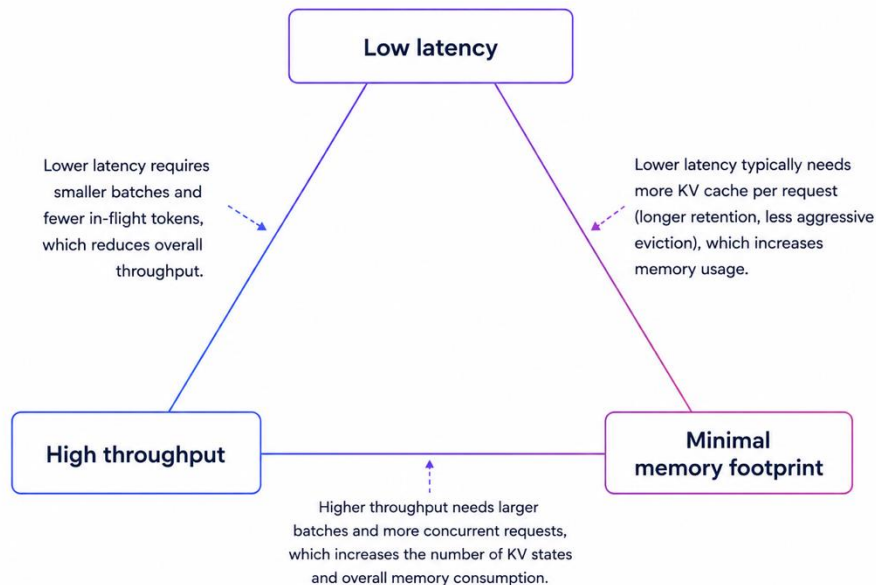
Redis + GCRA
RPM / TPM + stream counters

Quota service
signed ticket + idempotent reports

Regex / DFA + classifier
streaming buffer + audit store

Inference engine is the basement

- Minimize latency
- Maximize throughput
- Manage memory efficiently
- Allow controlled tradeoffs



Optimizing latency

	QUEUE	PREFILL	DECODE
Stage-specific knobs (what to optimize)	Reduce queueing Continuous batching Priority-aware scheduling Admission control	Reduce prefill work Prefix caching / KV reuse Chunked prefill P/D disaggregation	Reduce decode work per token Speculative decoding Quantization (W + KV) Parallelism
Cross-stage knobs (apply across prefill and decode)	Optimized kernels (e.g., FlashAttention variants) Quantization (weights + KV) Parallelism P/D disaggregation CUDA graphs		

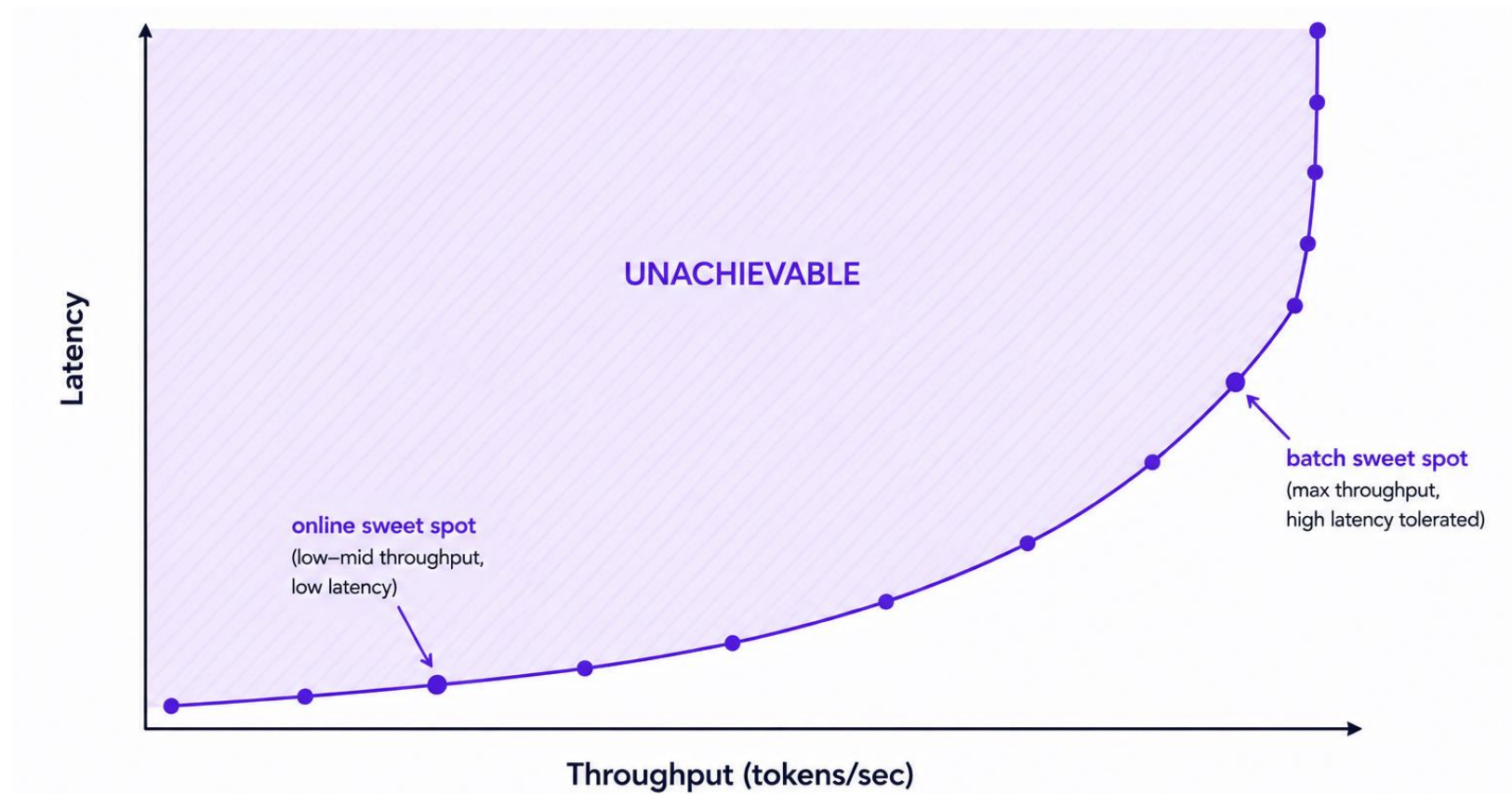
Optimizing throughput

	QUEUE	PREFILL	DECODE
Stage-specific knobs (what to optimize)	Maximize admission rate Continuous batching Large max_num_seqs / max_num_batched_tokens Fair scheduling	Maximize prompt tokens/sec Chunked prefill Prefix caching / KV reuse Large prefill batches	Maximize output tokens/sec Large decode batches High gpu_memory_utilization Output length caps
Cross-stage knobs (apply across prefill and decode)	Quantization (W + KV) Parallelism P/D disaggregation Optimized kernels (e.g., FlashAttention variants) CUDA graphs		

Managing memory

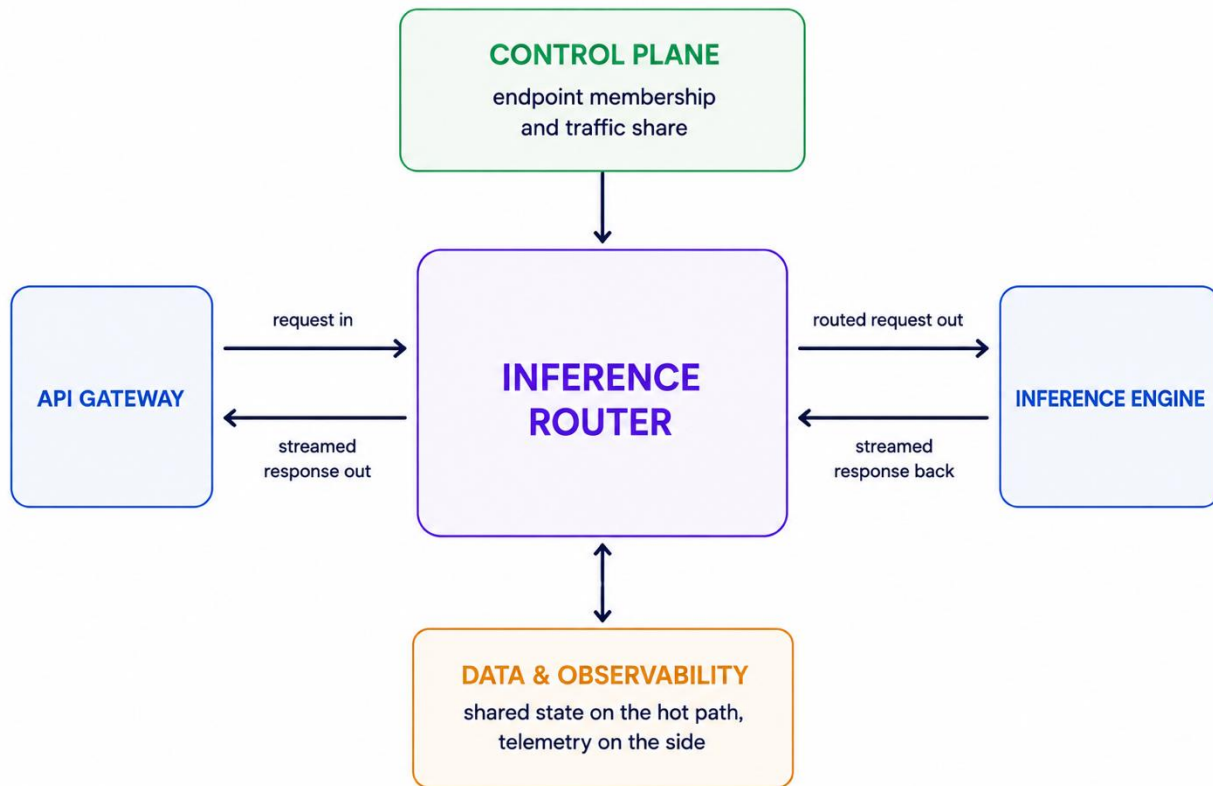
ALLOCATION	REUSE	COMPRESSION	MEMORY PRESSURE RELIEF
Maximize available memory High memory_utilization Block	Avoid allocating in the first place Prefix caching	Shrink what's stored Weight quantization KV cache quantization	Relieve memory pressure KV cache offloading Preemption + recompute Admission control Early rejection Backpressure

There is no free lunch



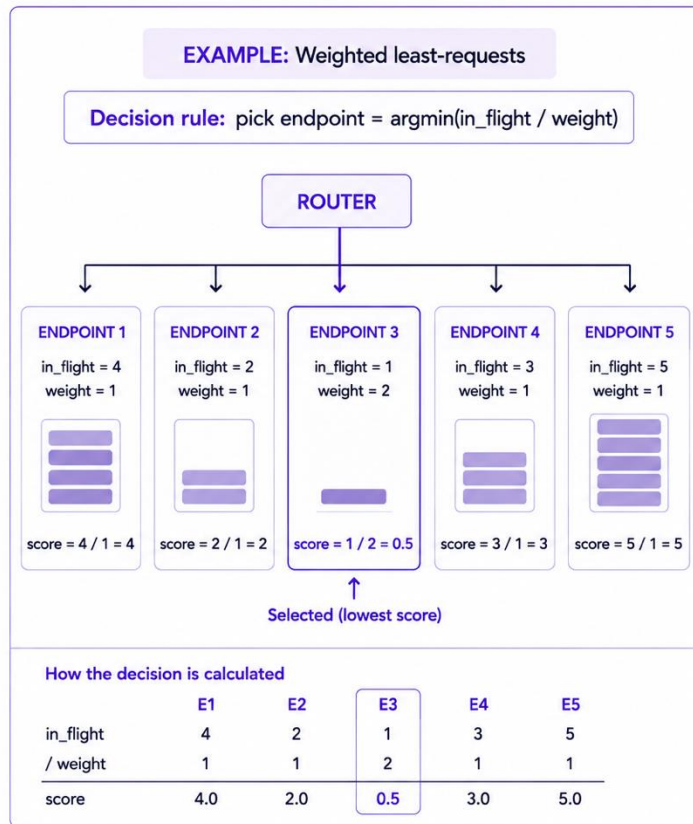
Questions?

Inference router makes it work at scale



Classic routing strategies

Algorithm	Decision rule (pick endpoint that...)
Round-robin / random	next endpoint (or random endpoint)
Least-requests / least-connections	$\min(\text{in_flight})$
Weighted least-requests	$\min(\text{in_flight} / \text{weight})$ = $\min(\text{in_flight} \times 1 / \text{weight})$
Consistent hashing	$\text{hash}(\text{session or caller key})$



SYSTEM: You are a helpful assistant that answers concisely.

USER: Summarize the following document in bullet points ...

- Shared prefix (likely common)

- ## 1. Prefix hashes

Compute hashes for multiple prefix lengths (long $\rightarrow$ short)

P_1 (longest)
P_2
P_3
$\vdots$
P_{16} (shortest)

lookup
→

- ## 2. Prefix $\rightarrow$ endpoint index (example)

For each prefix hash, get endpoints that recently served that prefix (most recent first)

Prefix hash	Warm endpoints (most recent first)	Last seen
P_1	E1, E2, E3	10s ago (E3)
P_2	E2, E3	8s ago (E3)
P_3	E3	5s ago (E3)
...	...	...
P_{16}	—	—

- ### 3. Filter candidates

Keep only endpoints that are healthy,
below overload threshold, and have
enough free capacity (KV, slots, etc.)

Endpoint	Healthy	Load (vs. threshold)	Free KV capacity
E1	✗	—	—
E2	✓	High	OK
E3	✓	OK	OK

E3 is the first viable match at the longest prefix (P_1).

- #### 4. Routing decision

Route request
to E3

Why E3?

- ✓ Longest matching prefix (P_1)
- ✓ Most recently warmed
- ✓ Healthy
- ✓ Load and capacity OK

Endpoint E3 (target)

KV cache in HBM



Prefix P_1 already cached

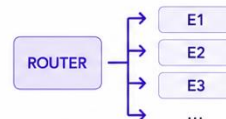
Skip prefill for this prefix
→ start decoding immediately

5. Fallback if no viable candidate at any prefix level

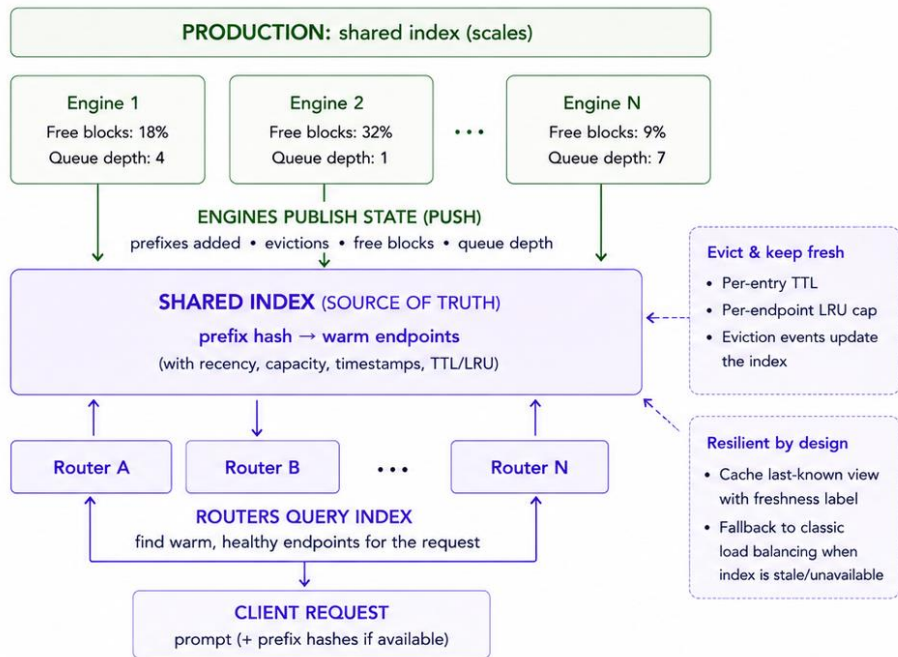
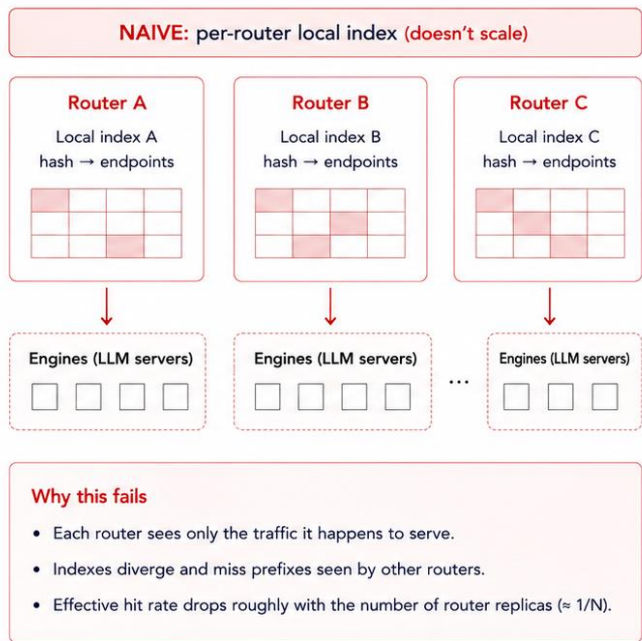
If no prefix level yields a viable endpoint, fallback to the operator's load balancer (e.g., least-loaded / weighted least-connections).

Examples:

- Least-loaded
- Weighted least-connections
- Round-robin (as last resort)

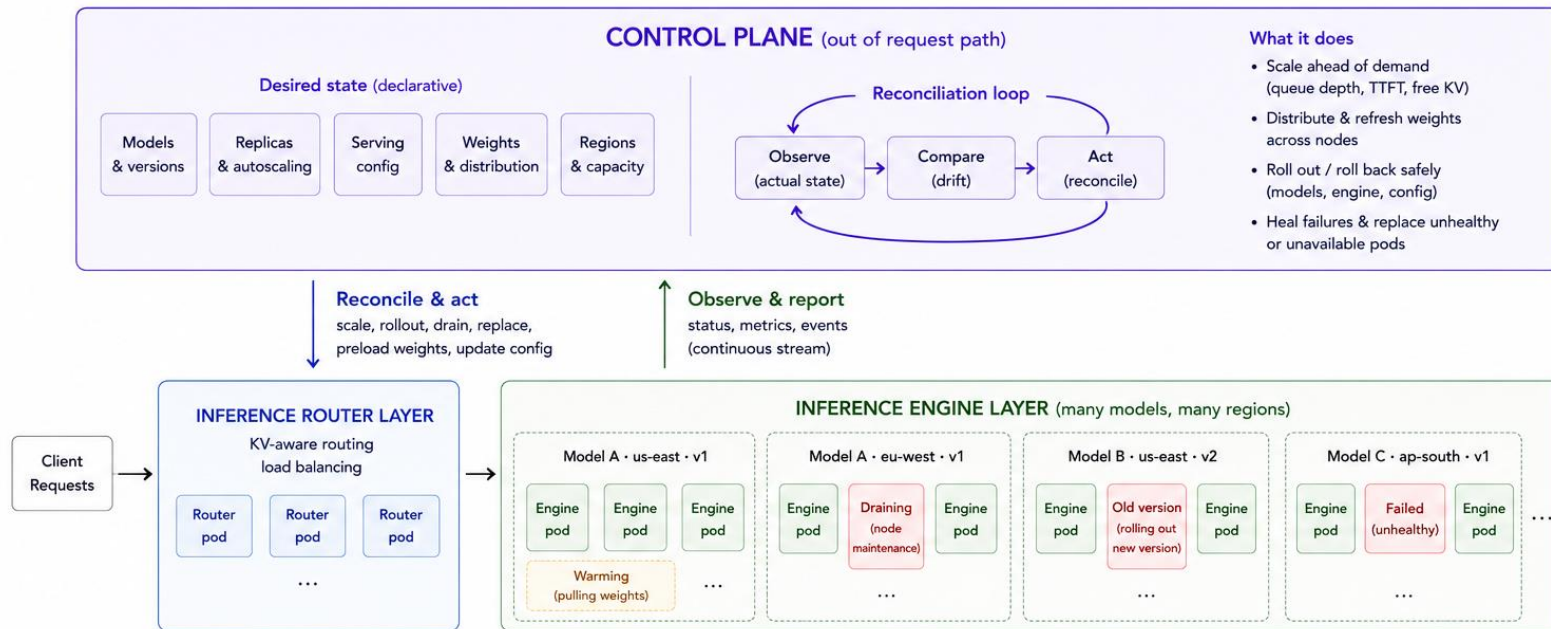


KV-aware routing: infrastructure



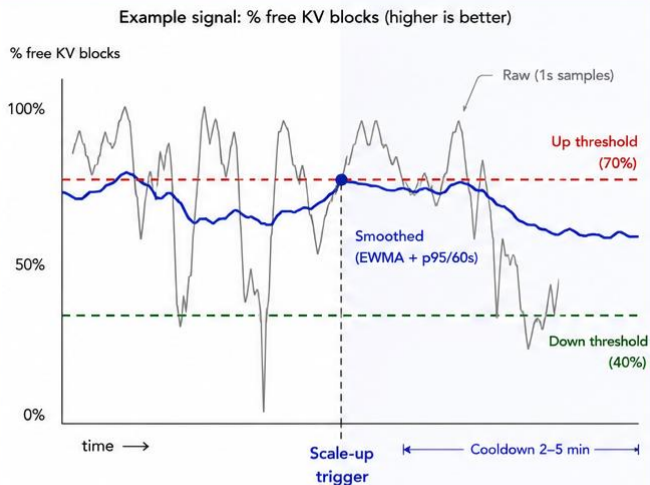
Questions?

Control plane: one plane to rule them all



Autoscaling and cold starts

1) When do we scale? → Smoothed, gated triggers



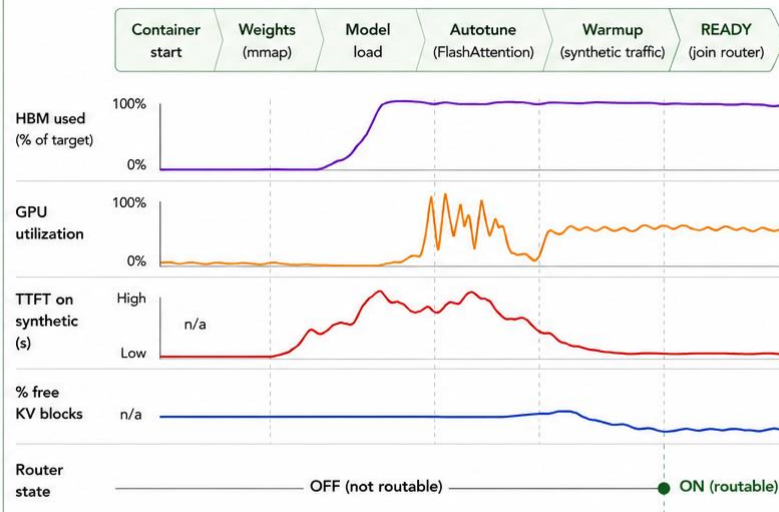
How we avoid flapping



We trigger once, then wait for the new capacity to arrive.

2) What happens after scaling fires? → Cold-start signature

Lifecycle of a new pod (alive early, routable late)



Readiness gate (examples)

- ✓ HBM at expected steady-state level
- ✓ TTFT (synthetic) below threshold for N reqs
- ✓ GPU util in healthy range, no errors
- ✓ Then the router turns it ON

Fighting cold starts

Three angles to reduce cold starts

FASTER

Minimize startup time

Tiered weight delivery network (Tier 1/2/3)

Content-addressed weight storage (dedupe across versions)

Deploy-time mirroring (no upstream dependency at start)

Persisted kernel autotune cache (15s → ~1s)

Snapshot / restore loaded engines (CRIU or vendor) (~10s restore)

EARLIER

Start before demand arrives

Predictive scale-up (diurnal, events, short horizon forecast)

Warm pools (N always-running idle replicas for top models / tiers)

SAFER

Expose only when stable

Router readiness gate until cold-start signature stabilizes (HBM, GPU%, synthetic-warmup TTFT)

Synthetic warmup across expected prompt-length distribution before opening to real traffic

Tiered weight delivery network (faster scale-up)

Tier 3

Mirrored origin (content-addressed blob storage)

Deploy-time mirror from HF / model registry

Parallel byte-range fetches, GB/s aggregate

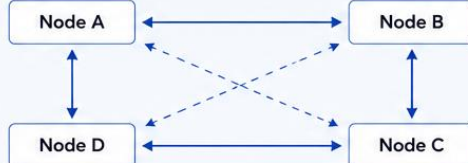
Cold burst: each node pulls only its assigned chunks in parallel

Tier 2

In-cluster peer cache (P2P via consistent hash ring)

Each node holds a deterministic subset of chunks

Peer-to-peer transfer over high-speed fabric



Burst scale-up:
origin traffic
stays ~1 ×
model size
(not N ×)

Tier 1

Node-local NVMe (multi-GB/s mmap)

Weights resident on every candidate node

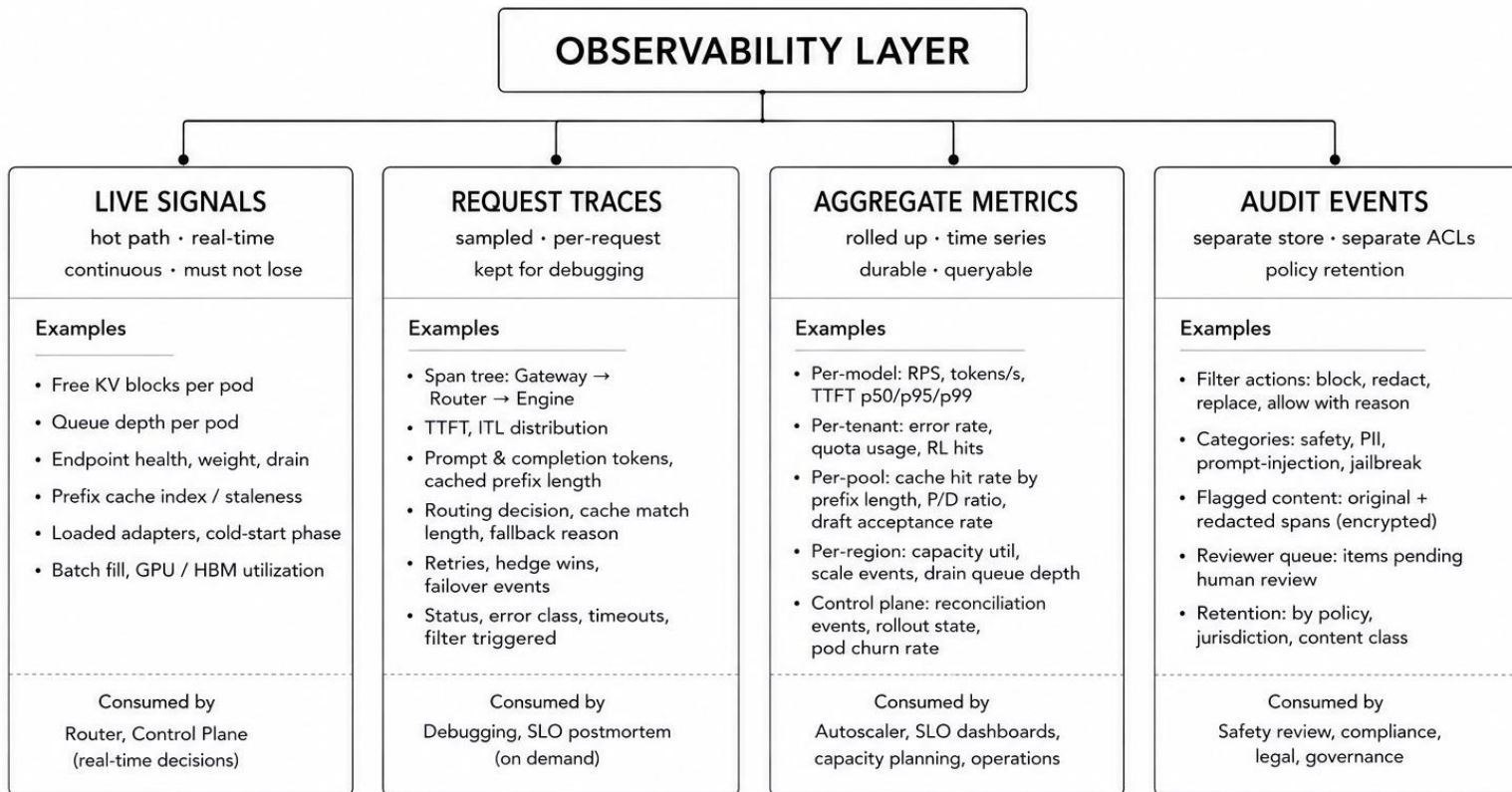


Questions?

Observability layer: design principles

Measure LLM-native signals	TTFT, ITL, KV pressure matter more than CPU. Examples: TTFT, ITL, free KV blocks, queue depth, prefix-cache hit rate, prefill share, batch fill ratio.
Attribute at write time	Identity propagates with every record. Examples: tenant • project • principal • model • engine version • region.
Keep traces unbroken	Requests cross runtimes without losing context. Examples: Python (Gateway) → Go (Router) → C++ (Engine). W3C trace context + per-layer span attributes.
Cardinality is a budget	Sample by class, not uniformly. Examples: enforce label cardinality budgets, keep errors and premium traffic, sample by shape (tails, long prompts).
Audit is a separate system	Content telemetry is isolated from operational telemetry. Examples: dedicated audit store, row-level ACLs, policy-driven retention.

What lives there

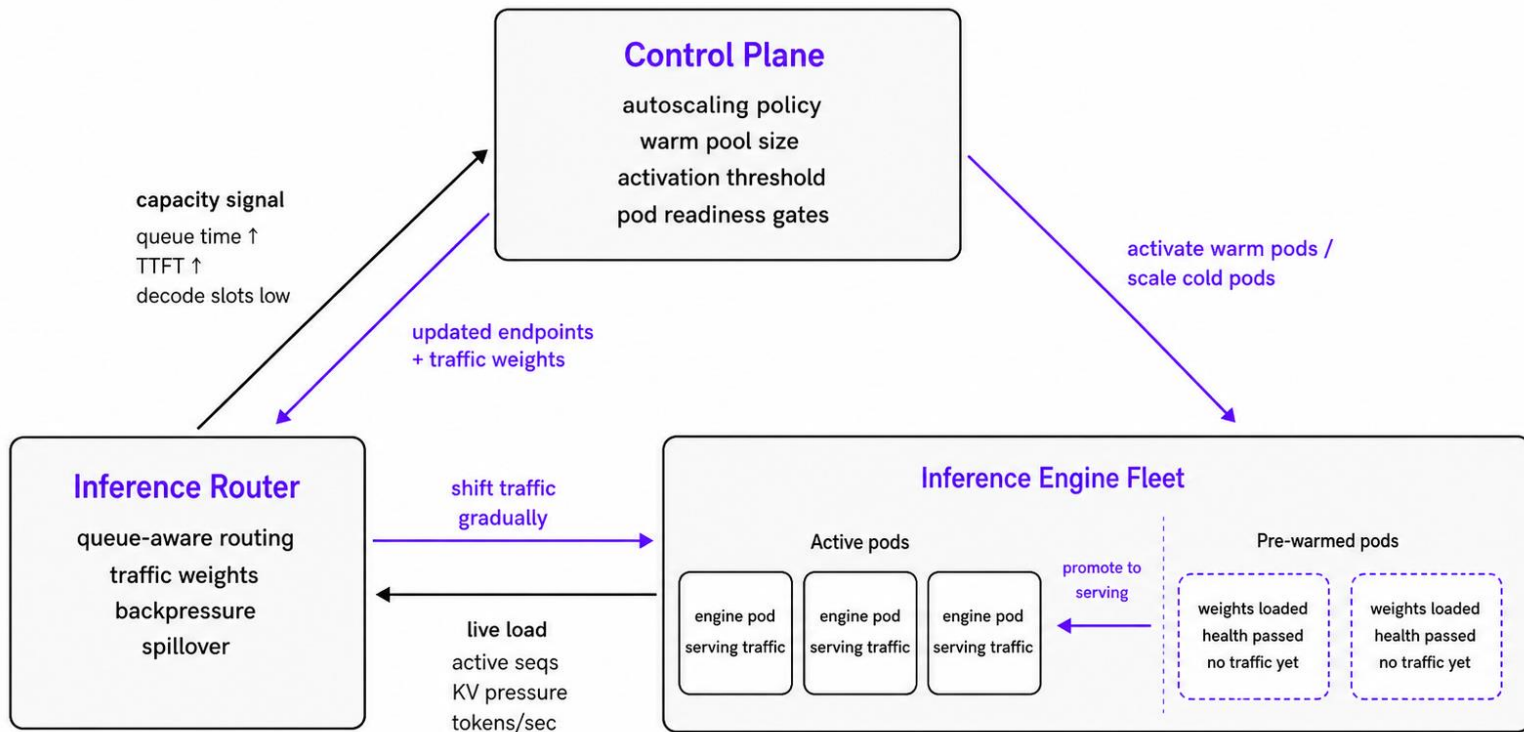


Questions?

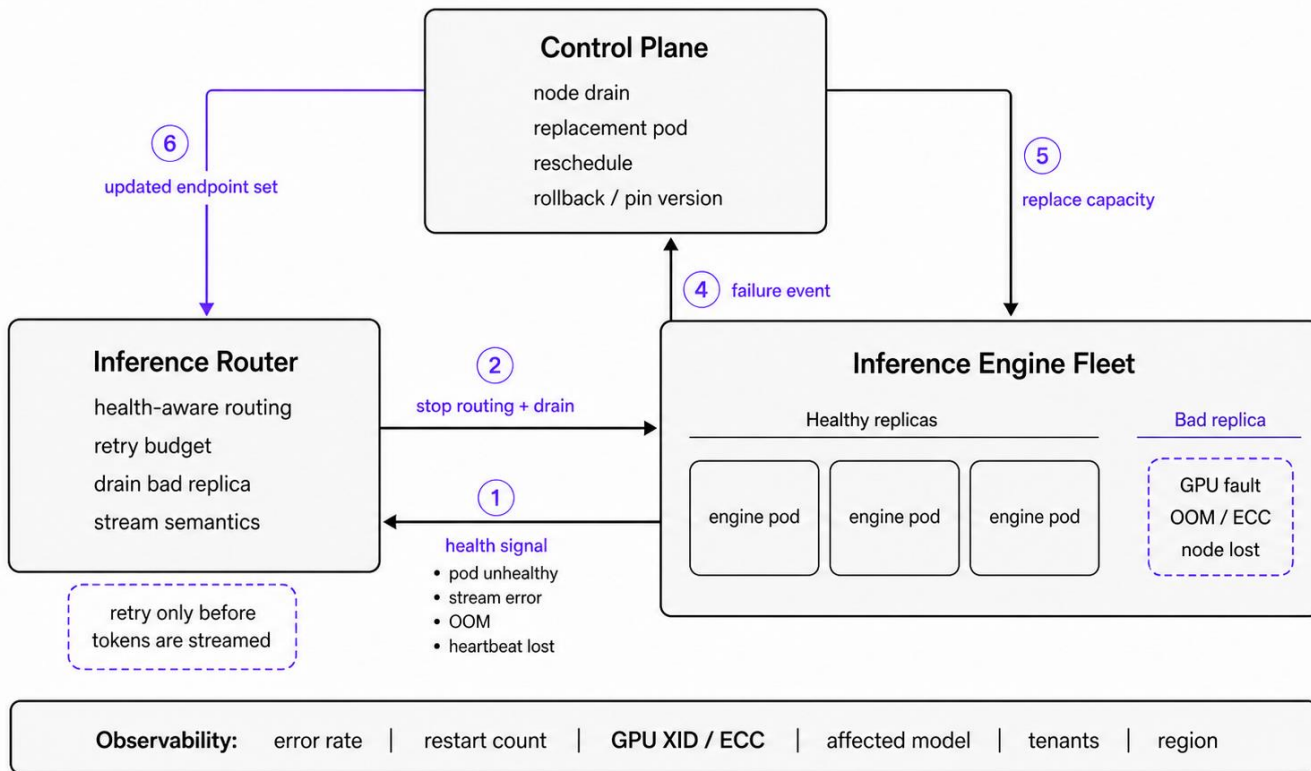
Section 3

How layers talk to each other

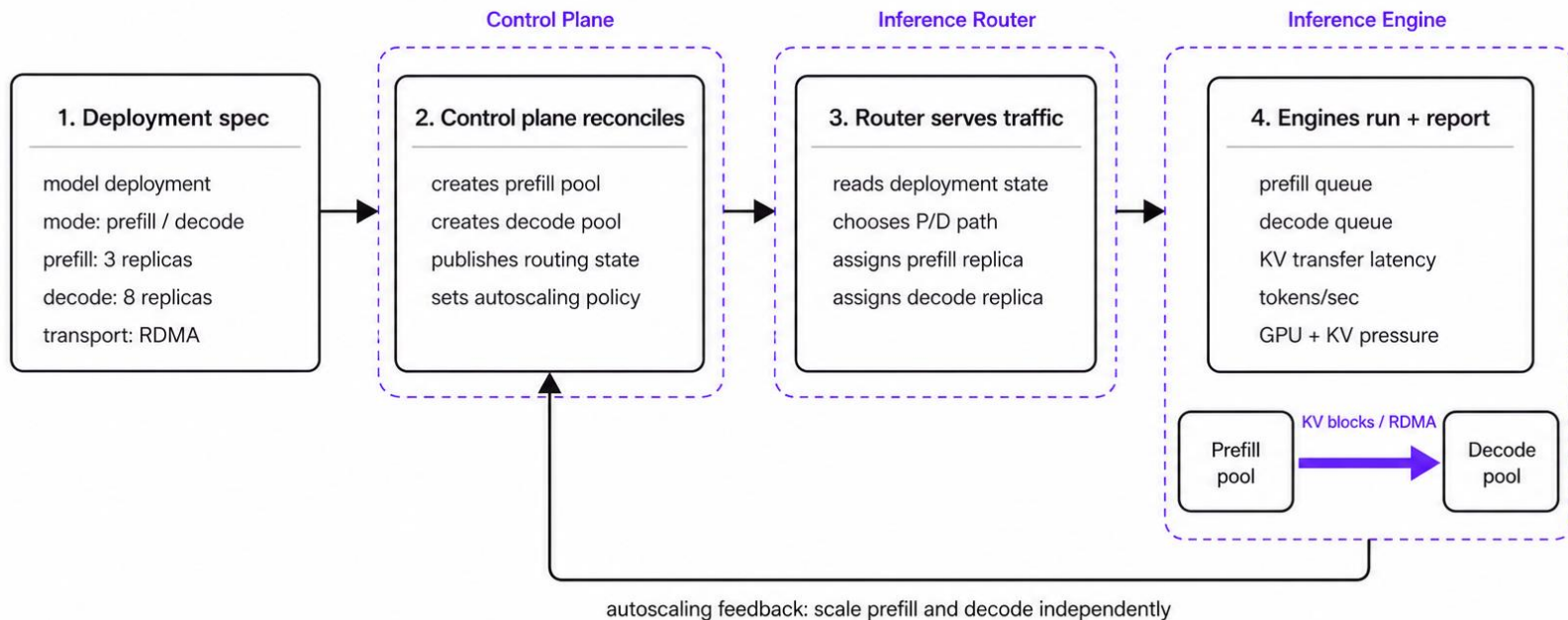
Practical example: traffic spike



Practical example: hardware incident



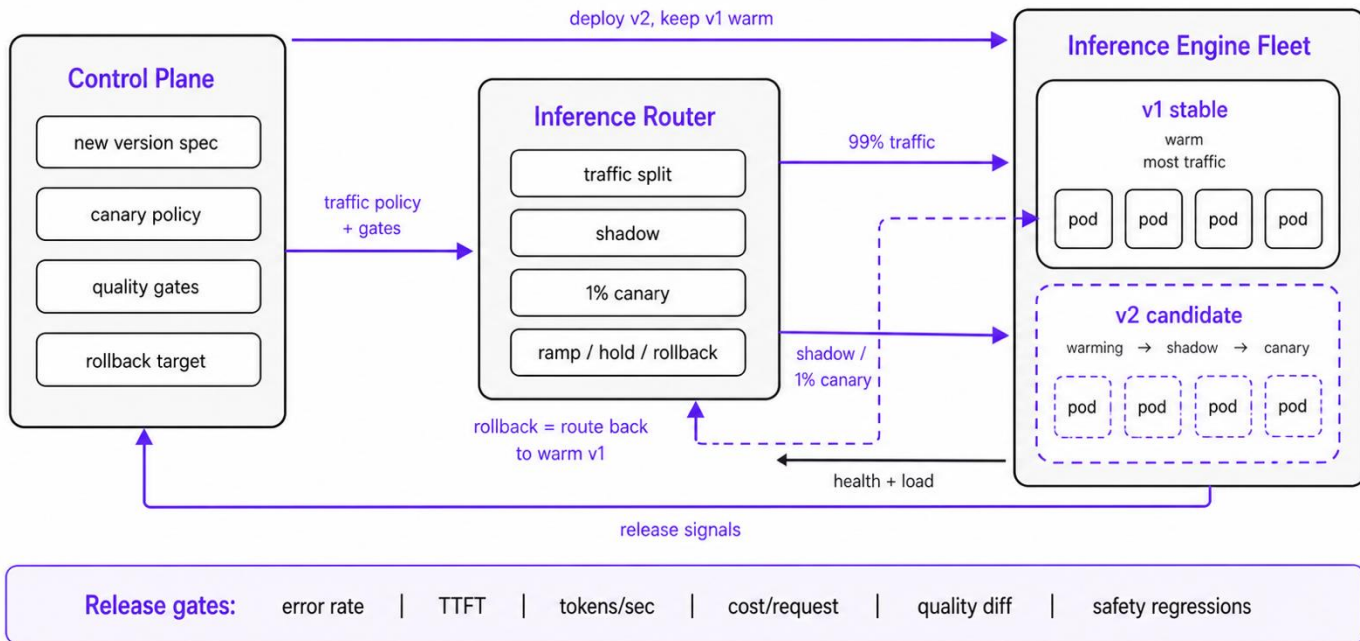
Practical example: P/D deployment lifecycle



Practical example: ~~Friday evening~~ rollout

Practical example: Friday evening rollout in production LLM inference

safe rollout with canary, shadow, and instant rollback



Questions?

Section 4

*Designing for reliability and
maintainability*

Perf gate

Load format	Pattern	Duration	Gate
Fixed-shape synthetic	short chat / long prompt / long decode	30 min each	p95 TTFT + TPS inside target
Ramp	25 -> 50 -> 75 -> 100 -> 125%	20 min / step	no queue runaway
Spike	0 -> target RPS instantly	30 min	backpressure + warm pool works
Replay	timestamped JSONL traffic	2-4 h	matches prod latency envelope
Soak	expected peak load	12-24 h	no leaks / restarts / KV fragmentation

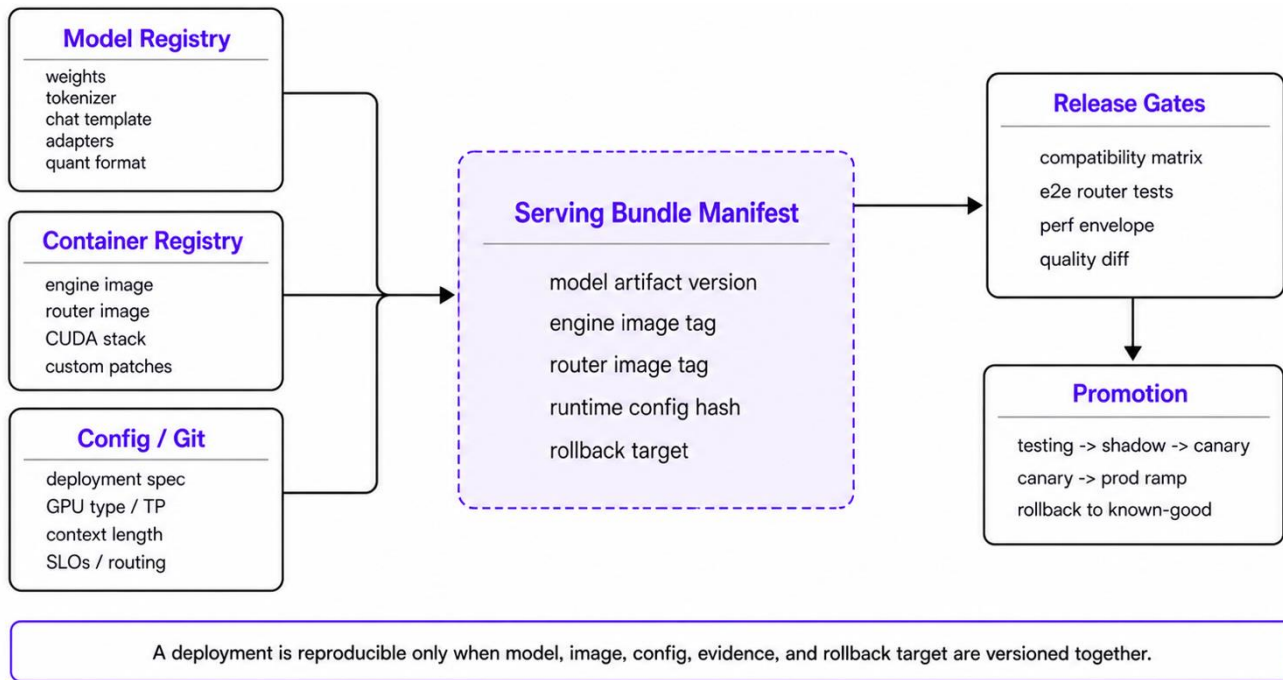
Infra: synthetic API caller | Kubernetes jobs | trajectory replay | AIPerf / Argo workflows

Quality gate

Gate	What it catches	Example infra / scope
Golden prompts	obvious regressions	fixed prompt set
Format gate	JSON / tool / schema breakage	parser checks
Benchmark gate	reasoning, code, math drift	OpenBench: AIME, GPQA, MMLU, MATH-500, HumanEval
Safety gate	refusal and jailbreak regressions	policy evals
Domain gate	customer-specific failures	Eval Scope / customer prompt packs
Pairwise diff	old vs new behavior change	previous known-good comparison

Output: quality diff | known regressions | explicit accept / reject decision

Think bundle



Thank you!